



Manual do Programador - Gestão de Utentes

Guia técnico para manutenção da área de Utentes no projeto central.

Ramo	Gestão de Utentes
Destinatários	Programadores e responsáveis técnicos
Repositório	https://github.com/MenteMovimento/central-mente-movimento
Site	https://central-mente-movimento.vercel.app
Atualização	22/06/2026

1. Visão geral da Central

A Central MenteMovimento junta os três ramos num único projeto publicado na Vercel e ligado a um único projeto Supabase. A autenticação é centralizada; os dados continuam separados por tabelas e contexto funcional.

- Repositório oficial: <https://github.com/MenteMovimento/central-mente-movimento>
- Site de produção: <https://central-mente-movimento.vercel.app>
- Branch principal: main.
- Build de produção: `npm run build`.
- Diretório publicado pela Vercel: `public`.
- Não publicar ficheiros `.env`, bases SQLite locais, anexos reais, exports com dados pessoais ou chaves privadas.

2. Variáveis e segurança

- `SUPABASE_URL` e `SUPABASE_ANON_KEY` podem ser usadas pelo frontend.
- `SUPABASE_SERVICE_ROLE_KEY` é secreta e deve ficar apenas em Environment Variables da Vercel ou ambiente local protegido.
- `SUPABASE_SECRET_KEY` é usada pelo backend Python de Utentes quando aplicável.
- Depois de mudar variáveis na Vercel, fazer redeploy.
- Se uma chave secreta for exposta, gerar uma nova no Supabase, atualizar Vercel e revogar a antiga.

3. Fluxo de alteração

- 1 Alterar o código localmente.
- 2 Executar `npm run build` na raiz.
- 3 Testar as páginas afetadas.
- 4 Copiar as alterações para a cópia ligada ao GitHub, se estiver a usar a pasta `.publish`.
- 5 Fazer commit com mensagem clara.
- 6 Fazer push para main.
- 7 Verificar o deployment automático na Vercel.

4. Estrutura do ramo Utentes

- Backend principal: `portal/modules/utentes/app.py`.
- Entrada serverless: `api/utentes-app.py`.
- Schema Supabase original: `portal/modules/utentes/supabase_schema.sql`.
- PDFs estáticos do manual: `portal/modules/utentes/docs/`.
- Tabelas principais: `utentes`, `utente_abas`, `historico`, `utente_anexos`.
- Bucket de anexos: `documentos-utentes`.

5. Separadores e persistência

- A lista base de utentes fica na tabela `utentes`.
- Separadores extensos são guardados em `utente_abas` por `utente_id` e nome do separador.
- Pagamentos e Mensalidades é um separador próprio e deve continuar entre Formulário de Referência e Informações em Caso de Emergência.
- Anexos PDF são gravados em `utente_anexos` e Storage quando Supabase está ativo.

- Histórico regista operações de criação, edição, estado, anexos e eliminação.

6. Cuidados técnicos

- A função Python em Vercel não deve depender de ficheiros locais persistentes.
- SQLite serve apenas para desenvolvimento local.
- Ao alterar HTML gerado em app.py, testar navegação, gravação e retorno aos separadores.
- Campos clínicos e dados pessoais exigem cuidado com logs, screenshots e exports.
- Manter traduções PT/EN quando adicionar texto de interface.

Checklist técnica antes de publicar

- 1 Confirmar que as alterações estão na pasta local correta.
- 2 Executar o gerador de manuais quando houver mudança funcional.
- 3 Executar npm run build na raiz do projeto.
- 4 Abrir pelo menos login, dashboard e o ramo alterado.
- 5 Testar navegação entre Sócios, Utentes e Dispositivos.
- 6 Confirmar que tema escuro, idioma e menu mantêm comportamento global.
- 7 Fazer commit e push para main apenas depois dos testes básicos passarem.

Ficheiros principais deste ramo

- portal/modules/utentes/app.py - backend, HTML gerado, rotas e persistência.
- api/utentes-app.py - entrada serverless usada pela Vercel.
- portal/modules/utentes/docs/ - manuais PT/EN descarregados pelo site.
- portal/modules/utentes/supabase_schema.sql - schema original do ramo.
- scripts/prepare-vercel-output.mjs - cópia do ramo para a pasta public.

Dados, tabelas e Storage

- Tabela utentes guarda a ficha base e estado ativo/inativo.
- Tabela utente_abas guarda o conteúdo dos separadores por utente.
- Tabela utente_anexos e bucket documentos-utentes guardam PDFs associados.
- Histórico deve registar alterações de fichas, anexos, estado e eliminações.
- SQLite só deve ser usado localmente; produção deve usar Supabase.

Regras de compatibilidade

- Não alterar nomes de colunas em produção sem migração SQL e plano de reversão.
- Não remover campos usados por importação, exportação, estatísticas ou histórico.
- Manter a navegação com caminhos absolutos, por exemplo /area/socios/ e não caminhos relativos.
- Manter textos novos nas tabelas de tradução quando o conteúdo aparece na interface.
- Confirmar que o build da Vercel usa os mesmos ficheiros que foram testados localmente.

Diagnóstico de problemas em produção

- Se só a Vercel falhar, verificar logs do deployment e variáveis de ambiente.

- Se Supabase devolver erro de sessão, verificar SUPABASE_SERVICE_ROLE_KEY e permissões.
- Se uma API Python falhar, confirmar importações, caminhos de ficheiros e limites serverless.
- Se um PDF antigo continuar a abrir, limpar cache do browser ou confirmar se o ficheiro foi copiado para public.
- Se houver lentidão, confirmar chamadas repetidas, listas muito grandes e loops de renderização.

Política de dados reais

- Nunca commitar exports reais de sócios, utentes ou dispositivos.
- Nunca commitar anexos reais ou bases SQLite com dados pessoais.
- Usar dados fictícios em testes e prints de suporte.
- Se uma chave ou ficheiro sensível for publicado por engano, revogar, substituir e registar o incidente.